



Programul de studii: **CALCULATOARE**

Aplicatie de testare online

Adaugare/editare teste online (orice domeniu)
tip grila

Autor: Vancea Paul Alexandru

2025



Cuprins

1. Introducere	2
1.1. Context general	2
1.2. Obiective	2
1.3. Lista MoSCoW	3
1.4. Cazuri de utilizare	4
2. Arhitectura	5
2.1. Front-End React	5
2.2. Back-End Spring Boot	5
2.3. Comunicare între front-end și back-end	5
2.4. Baza de date MySQL	6
2.5. Beneficii ale acestei arhitecturii	6
3. Implementare Front-End	7
3.1. Stack Tehnologic	7
3.2. Structura și Navigația	7
3.3. Gestiunea Autentificării	8
3.4. Implementarea WebSocket (Client)	8
4. Implementare Back-End	9
4.1. Platformă și Module	9
4.2. Securitate și JWT	9
4.3. WebSocket și STOMP	10
4.4. API REST și Rute	11
4.5. Persistența Datelor	12
5. Concluzii	13
6. Bibliografie	14

1 Introducere

1.1 Context general

Proiectul își propune dezvoltarea unei aplicații web interactive, multi-utilizator, care să realizeze sesiuni de întrebări și răspunsuri de tip quizz, cu monitorizare a rezultatelor în timp real. Nevoia de a oferi un mediu dinamic pentru evaluarea cunoștințelor și pentru colectarea feedback-ului instantaneu în timpul unei sesiuni de predare sau eveniment este scopul principal. Aplicația este concepută pentru a fi utilizată atât de un moderator/profesor cât și de un public larg (studenți/participanți).

Scopul principal este acela de a ușura procesul de interacțiune și evaluare rapidă a cunoștințelor, într-un mod accesibil și modern.

1.2 Obiective

Scopul acestei aplicații web este de a facilita interacțiunea didactică și evaluarea formativă. Prin dezvoltarea acestei aplicații se dorește la atingerea următoarelor obiective:

- Gestiunea completă a întrebărilor și sesiunilor de quizz de către utilizatorul cu rol de profesor.
- Efectuarea rapidă a quizz-urilor/răspunsurilor de către studenți.
- Afișarea în timp real a distribuției răspunsurilor (statistici) pe ecranul Profesorului, utilizând tehnologia WebSockets.
- Asigurarea securității datelor și a accesului bazat pe roluri (Profesor/Student) folosind JWT.
- Stocarea persistentă a întrebărilor, răspunsurilor și rezultatelor în MySQL.

1.3 Lista MoSCoW

În cadrul procesului de dezvoltare, stabilirea priorităților cerințelor este crucială. Metoda MoSCoW clasifică cerințele aplicației în patru categorii:

Table 1: Tabel 1.1 Lista MoSCoW

Must Have	Should Have
Login/Register & Roluri (Profesor/Student)	Afișarea scorului final individual (Student)
Operații CRUD pentru întrebare (Profesor)	Vizualizarea istoricului sesiunilor de quizz (Profesor)
Funcționalitatea de răspuns la întrebare (Student)	
Afișarea live a distribuției răspunsurilor (WebSockets)	

Could Have	Won't Have
Timer de expirare a sesiunii de quizz	Întrebări asociate fiecărui admin
Opțiuni de personalizare a temei (dark/light mode)	Integrare cu platforme de e-mail
Export de rapoarte (ex: CSV)	

1.4 Cazuri de utilizare

Această aplicație este destinată atât Profesorilor cât și Studenților. Cazurile de utilizare ilustrează modul în care cele două tipuri de utilizatori interacționează cu aplicația.

ADMIN (Profesor)

- Logare/Delogare
- Gestiune întrebări (CRUD: Adaugă, Modifică, Șterge)
- Gestiune sesiuni de Quizz (Creare sesiune, Activare, Închidere)
- Vizualizare rezultate **LIVE** (Distribuția răspunsurilor)
- Vizualizare scoruri finale (după încheierea sesiunii)

GUEST (Student)

- Logare/Delogare
- Efectuare Quizz Online (Răspuns la întrebări)
- Vizualizare scor personal (după corectare/finalizarea sesiunii)

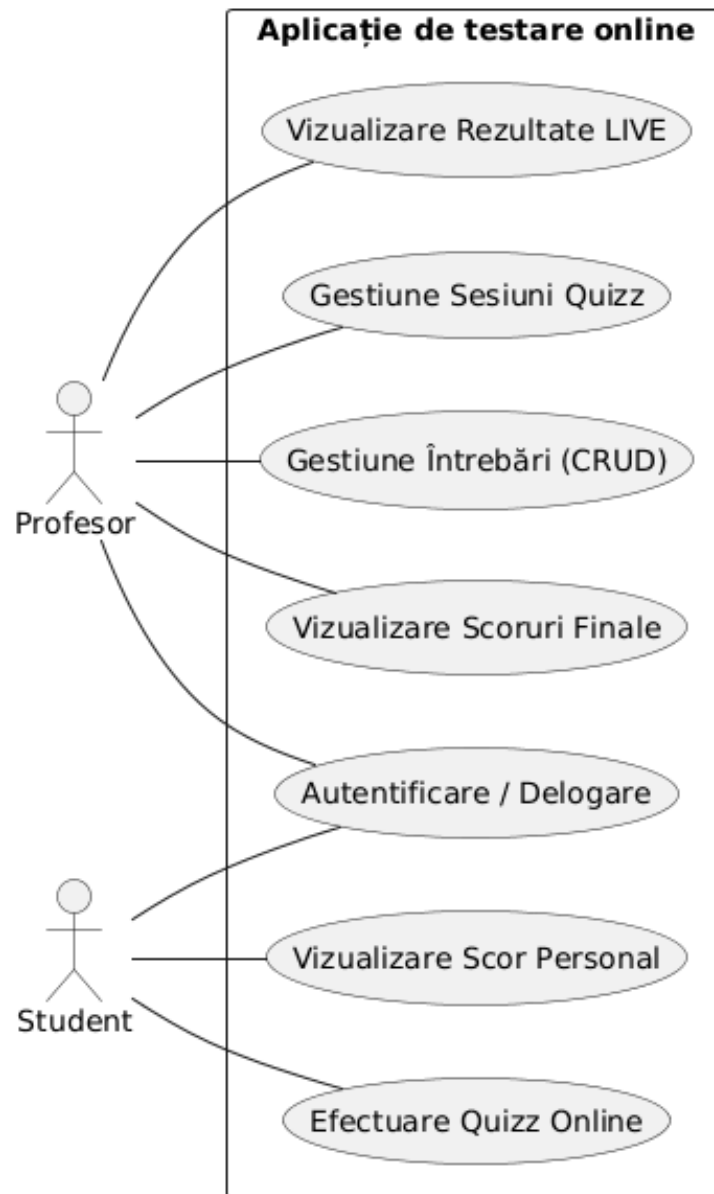


Figure 1: Figura 1.1 Diagrama cazurilor de utilizare

2 Arhitectura

Arhitectura aplicației este concepută pentru a asigura o performanță de înaltă calitate, scalabilitate și o interfață utilizator prietenoasă. Cu un front-end dezvoltat în React și un backend bazat pe Spring Boot, această arhitectură modernă îmbină tehnologii de ultimă generație pentru a oferi o experiență coerentă și robustă utilizatorilor.

Arhitectura Aplicației "Aplicație de testare online"

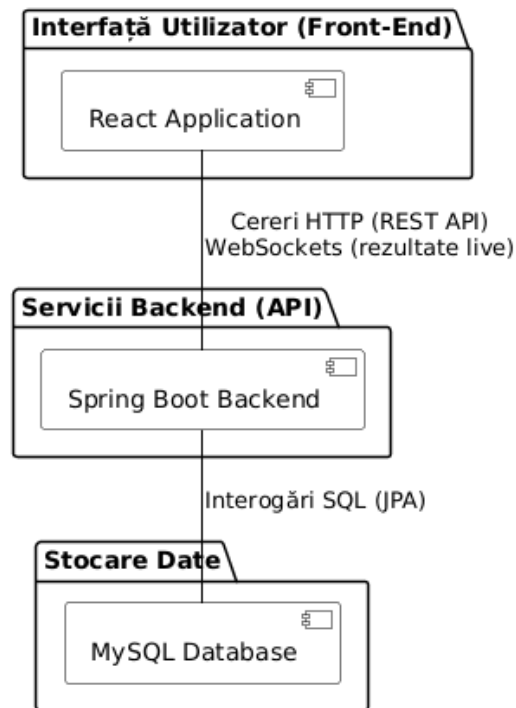


Figure 2: Figura 2.1 Arhitectura aplicației

2.1 Front-End React

Partea de frontend este dezvoltată în React, pentru a beneficia de modularitate, flexibilitate și viteză de dezvoltare. Componentele React vor gestiona starea aplicației și vor facilita interacțiunea fluidă a utilizatorului.

2.2 Back-End Spring Boot

Spring Boot reprezintă coloana vertebrală a back-end-ului, oferind un cadru robust pentru dezvoltarea serviciilor RESTful. Acesta va gestiona logica de

business, securitatea, și comunicarea cu baza de date prin Spring JPA.

2.3 Comunicare între Front-End și Back-End

Comunicarea clasică (CRUD) se va realiza prin intermediul cererilor HTTP, cel mai probabil utilizând o bibliotecă precum Axios. Comunicarea în timp real (pentru actualizarea rezultatelor live) se va realiza prin **WebSockets** (Spring WebSockets / STOMP).

2.4 Baza de Date MySQL

MySQL este ales ca sistem de gestionare a bazelor de date datorită popularității sale, ușurinței în utilizare și performanței dovedite în aplicațiile web. Interfața Spring Data JPA simplifică interacțiunea cu baza de date.

2.5 Beneficii ale acestei arhitecturii

- **Timp Real:** Integrarea WebSockets permite actualizarea instantanee a rezultatelor Profesorului, îmbunătățind interactivitatea.
- **Securitate:** Utilizarea Spring Security și JWT asigură o metodă robustă și *stateless* de gestionare a autentificării și autorizării.
- **Eficiență în dezvoltare:** Utilizarea framework-urilor populare (React și Spring Boot) contribuie la o dezvoltare rapidă și ușoară.
- **Scalabilitate:** Arhitectura permite extinderea facilă a capacităților aplicației pe măsură ce numărul de utilizatori crește.

3 Implementare Front-End

3.1 Stack Tehnologic

Interfața utilizator este construită utilizând **React 18 (18.3.x)**, o bibliotecă JavaScript modernă pentru construirea interfețelor reactive. Pentru gestionarea proiectului și serverul de dezvoltare, s-a optat pentru **Vite 5.x**, care oferă timpi de pornire foarte rapizi.

Principalele biblioteci utilizate sunt:

- **React Router DOM (6.28.x)**: Pentru navigația tip SPA (Single Page Application) și protecția rutelor în funcție de rol.
- **Axios (1.7.x)**: Client HTTP pentru comunicarea cu REST API-ul backend-ului. Include interceptori pentru atașarea automată a token-ului JWT.
- **@stomp/stompjs (7.0.x) & sockjs-client**: Pentru comunicarea în timp real prin WebSockets.

3.2 Structura și Navigația

Aplicația este structurată în pagini distincte, accesibile în funcție de rolul utilizatorului:

- **Pagini publice**: Login, Register.
- **ProfessorDashboard**: Permite crearea sesiunilor, activarea lor și vizualizarea statisticilor live. Include componenta **LiveStatsPanel**.
- **StudentDashboard**: Permite introducerea codului de acces, participarea la quizz și vizualizarea scorului final.

3.3 Gestiunea Autentificării

Starea de autentificare este gestionată printr-un **AuthContext**. Token-ul JWT (JSON Web Token) primit de la server este stocat în **sessionStorage**. Acest lucru permite o funcționalitate esențială: **izolarea sesiunilor pe tab-uri**. Un utilizator poate fi logat ca Profesor într-un tab și ca Student în altul, facilitând testarea și demonstrațiile.

3.4 Implementarea WebSocket (Client)

Pentru actualizarea în timp real a rezultatelor pe dashboard-ul profesorului, aplicația folosește protocolul STOMP peste SockJS.

- Frontend-ul se abonează la topicuri specifice sesiunii: `/topic/sessions/{sessionId}/questions/{ques`
- Când un student trimite un răspuns prin REST, backend-ul emite un eveniment pe WebSocket, iar graficele profesorului se actualizează instantaneu fără refresh.

4 Implementare Back-End

4.1 Platformă și Module

Backend-ul este dezvoltat în **Java 17** folosind framework-ul **Spring Boot 3.3.3**. Arhitectura respectă modelul stratificat clasic: Controller → Service → Repository → Entity.

Modulele principale Spring utilizate sunt:

- **spring-boot-starter-web**: Pentru expunerea serviciilor REST.
- **spring-boot-starter-security**: Pentru securizarea endpoint-urilor.
- **spring-boot-starter-data-jpa**: Pentru interacțiunea cu baza de date prin Hibernate.
- **spring-boot-starter-websocket**: Pentru funcționalitățile de timp real.
- **spring-boot-starter-validation**: Pentru validarea datelor de intrare (ex: @NotBlank, @Email).

4.2 Securitate și JWT

Securitatea este implementată folosind un filtru custom (**JWT Authentication Filter**). Aplicația este **stateless** (**SessionCreationPolicy.STATELESS**), bazându-se exclusiv pe token-ul JWT pentru identificare.

Biblioteca folosită pentru JWT este **jjwt (0.11.5)**. Token-ul conține în payload rolul utilizatorului (**PROFESSOR** sau **STUDENT**), ceea ce permite autorizarea la nivel de metodă sau rută.

Politici de acces configurate:

- **Public**: `/api/auth/register`, `/api/auth/login`, `/ws/**` (pentru handshake-ul inițial WebSocket).
- **Profesor**: `/api/questions/**`, `/api/professor/**`.
- **Student**: `/api/student/**`.

4.3 WebSocket și STOMP

Serverul configurează un endpoint WebSocket la adresa `/ws` cu suport pentru SockJS (fallback pentru browserele vechi). Mesajele sunt transmise folosind un broker simplu în memorie.

Fluxul de date real-time:

1. Studentul trimite răspunsul prin POST HTTP.
2. Serviciul `QuizSessionServiceImpl` salvează răspunsul în DB.
3. Același serviciu calculează noile statistici și le trimite prin `SimpMessagingTemplate` către topicul public la care este abonat profesorul.

4.4 API REST și Rute

API-ul este organizat în jurul resurselor principale:

Autentificare:

```
1 POST /api/auth/register - Inregistrare utilizator
2 POST /api/auth/login    - Autentificare si generare token
3 GET  /api/auth/profile  - Detalii profil curent
```

Zona Profesor:

```
1 POST /api/questions      - Creare intrebare
2 POST /api/professor/sessions - Creare sesiune noua
3 POST /api/professor/sessions/{id}/activate - Pornire sesiune
   (generare cod)
4 GET  /api/professor/sessions/{id}/scores - Clasament
   studenti
```

Zona Student:

```
1 GET  /api/student/sessions/{code} - Join sesiune (cu
   shuffle intrebari)
2 POST /api/student/sessions/{code}/answers - Trimitere raspuns
3 GET  /api/student/sessions/{code}/score - Vizualizare
   rezultat final
```

4.5 Persistența Datelor

Sistemul de gestiune a bazelor de date este **MySQL 8.0**. Structura bazei de date este generată și actualizată automat în faza de dezvoltare prin proprietatea `hibernate.ddl-auto: update`.

Pentru a asigura integritatea datelor în timpul testelor, studenții primesc întrebările și opțiunile de răspuns într-o ordine aleatorie (shuffle) la momentul accesării sesiunii, prevenind copierea rapidă.

5 Concluzii

Aplicația de testare online realizată îndeplinește cu succes obiectivele propuse, oferind o soluție modernă și robustă pentru evaluarea cunoștințelor.

Arhitectura decuplată (React + Spring Boot) permite o scalabilitate ușoară și o experiență de utilizare fluidă. Integrarea tehnologiei WebSockets aduce o valoare adăugată semnificativă, transformând procesul de evaluare dintr-o activitate statică într-una interactivă și dinamică.

Securitatea implementată prin JWT și izolarea sesiunilor la nivel de tab demonstrează o atenție sporită la detalii și la scenariile reale de utilizare (ex: testarea aplicației de pe același dispozitiv).

6 Bibliografie

- 3.1. Spring Boot Documentation, <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- 3.2. React Documentation, <https://react.dev/>
- 3.3. STOMP Protocol Specification, <https://stomp.github.io/>
- 3.4. MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- 3.5. JSON Web Token (JWT) Introduction, <https://jwt.io/introduction>